

Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут» імені Ігоря Сікорського
Факультет інформатики та обчислювальної техніки
Кафедра автоматики та управління в технічних системах

Лабораторна робота №7

Тема: Патерни проектування

Виконав:
студент групи ІА-34
Єресько І.С
Перевірив:
Мягкий М.Ю

Київ-2025

Короткі теоретичні відомості

Шаблон «Mediator»

Шаблон «Посередник» застосовується тоді, коли необхідно організувати взаємодію між об'єктами через окремий керуючий елемент, замість того, щоб кожен із них зберігав посилання на всі інші. На відміну від «Команди», яка описує окрему дію, «Посередник» зосереджується саме на логіці взаємозв'язків між компонентами. Він стає корисним у випадках, коли у системі багато елементів, що взаємодіють складним і важкокерованим способом. У таких ситуаціях уся логіка обміну інформацією переноситься в окремий об'єкт, а самі компоненти лише звертаються до нього, не знаючи одне про одного. Цю роль можна порівняти з диригентом оркестру, який керує кожним інструментом і забезпечує узгоджену роботу всього ансамблю.

Проблема виникає тоді, коли у застосунку є складна форма з великою кількістю графічних елементів, що залежать один від одного. Будь-яка дія в одному компоненті може вимагати зміни інших частин інтерфейсу, що створює хаотичну та заплутану мережу зв'язків. Посередник пропонує винести весь механізм взаємодії в окремий клас, до якого звертаються всі елементи. Компоненти перестають знати про існування один одного, працюють лише через медіатора, а той, у свою чергу, координує необхідні зміни.

Перевага такого підходу полягає в тому, що код стає зрозумілішим, простішим для супроводу та розширення. Можна додавати нові посередники, не змінюючи вже існуючі компоненти, а зменшення кількості прямих залежностей збільшує гнучкість системи. Водночас є ризик, що з часом медіатор перетвориться на надмірно складний елемент, який бере на себе занадто багато логіки.

Шаблон «Facade»

Шаблон «Фасад» створює єдиний зрозумілий та спрощений спосіб доступу до складної підсистеми, приховуючи її внутрішній вміст. У великих підсистемах може бути багато класів та методів, і прямий доступ до них часто створює надмірну кількість зв'язків, що ускладнює структуру програмного коду. Фасад дозволяє виділити один узагальнений інтерфейс, через який клієнт працює з усією підсистемою, не знаючи про її внутрішню

устроювання. Це також дає змогу змінювати внутрішню реалізацію без потреби повідомляти всіх користувачів або змінювати їхній код.

Проблема стає очевидною в ситуації, коли компонент працює з різними протоколами та типами endpoint, а його внутрішні механізми стають настільки складними, що інструкція використання отримує надмірну кількість деталей. Інші системи залежать від внутрішньої структури, тому будь-які зміни її ламають. Фасад дозволяє створити один публічний клас із чіткими методами налаштування й виконання операцій, приховуючи всі внутрішні механізми. У результаті зовнішній код взаємодіє лише з простим інтерфейсом, а внутрішні зміни можна впроваджувати вільно та безболісно.

Переваги фасаду полягають у повній інкапсуляції підсистеми та створенні зручного, зрозумілого інтерфейсу. Недоліком є те, що приховування внутрішньої структури може обмежити можливості тонкого налаштування.

Шаблон «Bridge»

«Міст» використовується для поділу абстракції та конкретної реалізації так, щоб їх можна було змінювати окремо одна від одної. Це особливо корисно в тих випадках, де існує кілька різних типів абстракцій, які можуть по-різному реалізовуватися, що спричиняє вибухове зростання кількості підкласів у традиційній ієрархії.

Уявімо розробку графічного редактора, який має відображати фігури як на екрані, так і на принтері. Якщо реалізувати це традиційно, доведеться створювати підкласи для кожної фігури й для кожного способу відображення: окремо «лінія для екрана», «лінія для друку», «коло для екрана», «коло для друку» тощо. Додавання нового типу драйвера або нової фігури збільшуватиме кількість класів у геометричній прогресії. Шаблон «Міст» дозволяє створити дві незалежні ієрархії: одну — для фігур, іншу — для відображення. Фігури делегують процес малювання об'єкту графічного драйвера, який відповідає за конкретний спосіб відображення. Додавання нового драйвера або нової фігури не впливає на другу ієрархію.

Такий підхід робить систему значно гнучкішою та зрозумілішою, полегшує супровід, але водночас збільшує кількість абстракцій, що може ускладнити загальну структуру.

Шаблон «Template Method»

«Шаблонний метод» дає можливість винести загальний алгоритм у базовий клас, а окремі його частини залишити для перевизначення в підкласах. Це дозволяє повторно використовувати основну логіку та змінювати лише ті фрагменти, які відрізняються залежно від конкретного випадку. Як приклад можна уявити процес формування веб-сторінки, де структура однакова, а конкретне наповнення залежить від реалізації.

Проблема виникає тоді, коли один і той самий алгоритм намагаються адаптувати для багатьох форматів у межах одного класу, що призводить до численних умовних конструкцій і швидко робить код важким для читання. Коли для MPEG-4 додається підтримка MPEG-2, потім MPEG-1, а згодом нового формату, алгоритм ускладнюється настільки, що робота з ним стає неефективною. Шаблонний метод пропонує виділити незмінну частину алгоритму в базовий клас, а специфічні кроки передати підкласам, які перевизначають їх згідно зі своїми потребами. Завдяки цьому основний підхід залишається спільним, а підкласи реалізують лише різницю.

Перевагою шаблонного методу є можливість легкого повторного використання коду, однак він жорстко фіксує структуру алгоритму та може порушити принцип підстановки Лісков у випадку неправильного перевизначення методів. Крім того, якщо алгоритм стає надто великим, підтримувати таку систему стає складно через значну кількість віртуальних методів.

Тема: Патерни проектування.

Мета: Мета: Вивчити структуру шаблонів «Mediator», «Facade», «Bridge», «Template method» та навчитися застосовувати їх в реалізації програмної системи.

Хід роботи

8. Powershell terminal (strategy, command, abstract factory, bridge, interpreter, client-server)

Термінал для powershell повинен нагадувати типовий термінал з можливістю налаштування кольорів синтаксичних конструкцій, розміру

вікна, фону вікна, а також виконання команд powershell і виконуваних файлів, а також працювати в декількох вікнах терміналу (у вкладках або одночасно шляхом розділення вікна).

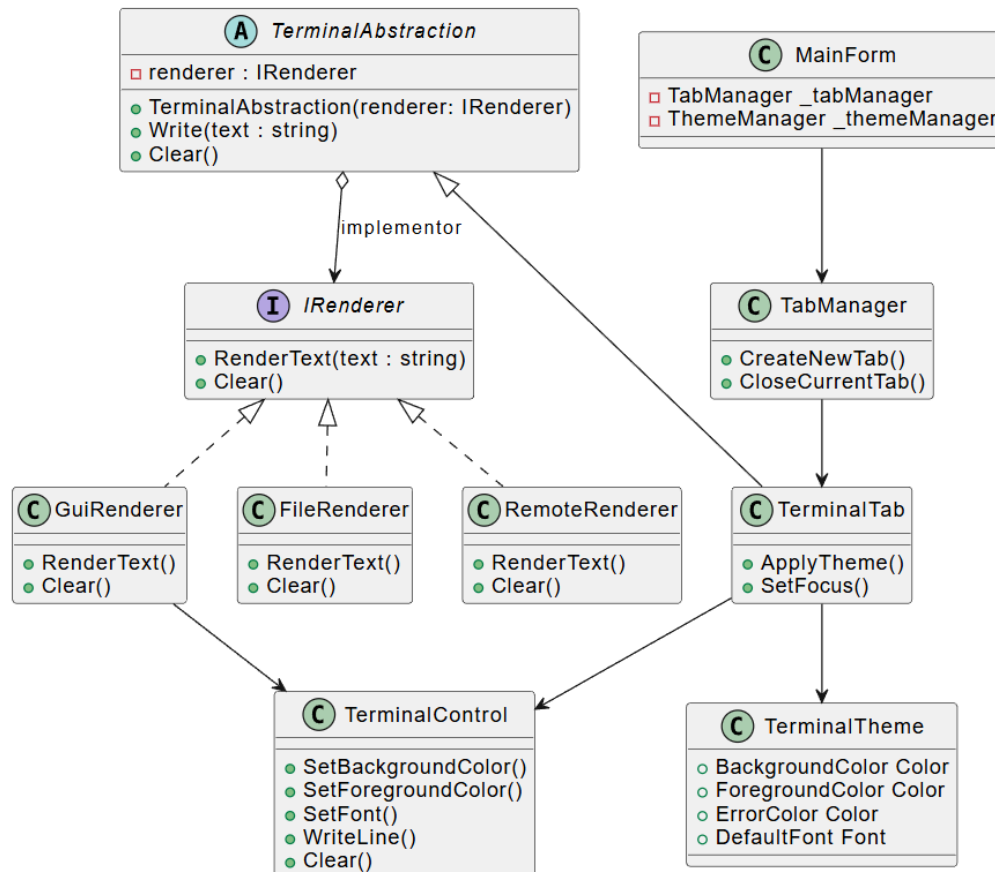


Рис 1: ClassDiagram (bridge)

[IRenderer](#)

[GuiRenderer](#)

[FileRenderer](#)

[RemoteRenderer](#)

[TerminalAbstraction](#)

[TerminalTab](#)

[TerminalControl](#)

[TerminalTheme](#)

[TabManager](#)

[MainForm](#)

У центрі реалізації знаходиться інтерфейс **IRenderer**, який представляє собою сторону реалізації. Він описує базові операції, необхідні для різних способів відображення даних: виведення тексту та очищення. Така абстракція дозволяє підключати будь-який тип виводу -графічний, файловий, мережевий без необхідності змінювати основну логіку терміналу. Три реалізації -**GuiRenderer**, **FileRenderer** та **RemoteRenderer** -показують приклади різних механізмів виводу. Перший працює з інтерфейсом користувача, другий записує інформацію у файл, а третій передає дані віддалено. Усі вони реалізують один і той самий інтерфейс, тому можуть бути взаємозамінними.

Абстракція представлена класом **TerminalAbstraction**, який містить посилання на реалізатор **IRenderer**. Цей клас інкапсулює основну поведінку терміналу: запис тексту та очищення. Важливо, що **TerminalAbstraction** не знає, як саме відбувається відображення, — він лише делегує виклики відповідному рендереру. Це дозволяє зробити логіку терміналу універсальною і незалежною від конкретної технології виводу.

Розширена абстракція, **TerminalTab**, успадковує базовий клас і додає функції, притаманні окремій вкладці терміналу, зокрема застосування теми і встановлення фокусу. Завдяки використанню **Bridge** ця вкладка також працює з будь-яким рендерером, який передається їй через базовий абстрактний клас. Таким чином **TerminalTab** може одночасно використовувати графічний рендер у UI або, за іншої конфігурації, відправляти вивід у файл або мережу — не змінюючи жодного рядка коду.

У діаграмі також присутні існуючі класи системи, такі як **TerminalControl**, **TerminalTheme**, **TabManager** і **MainForm**. Вони демонструють контекст, у якому використовується Bridge. TerminalControl залишається окремим UI-компонентом, який може служити інструментом для GUI-рендерера. TerminalTheme керує зовнішнім виглядом вкладки, а TabManager та MainForm відповідають за створення нових вкладок і загальне управління інтерфейсом. Взаємодія TerminalTab із TerminalTheme та TerminalControl показує, що шаблон Bridge вбудовується в існуючу архітектуру, не порушуючи її структуру.

Висновок:

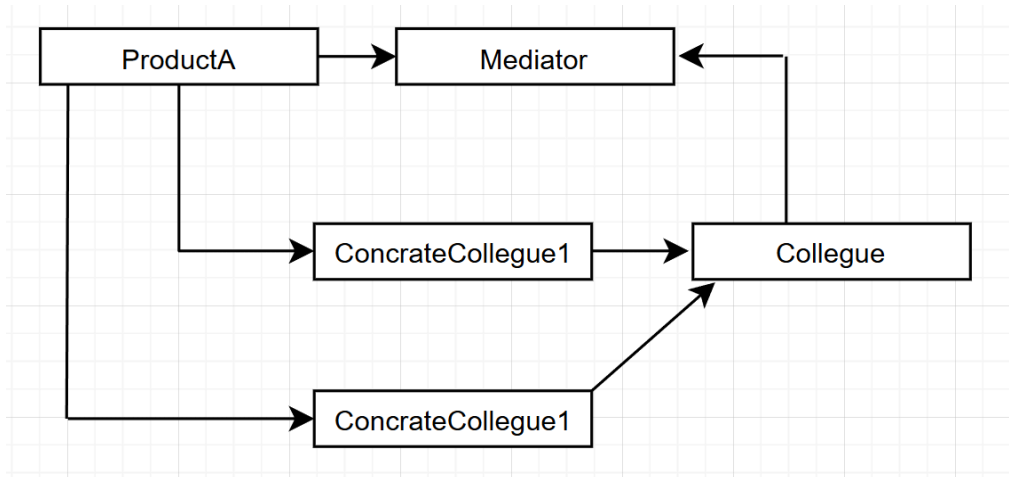
У процесі виконання роботи я ознайомився з патернами проектування «Composite», «Flyweight», «Interpreter» та «Visitor», розглянув їхню структуру, призначення та можливості застосування у складних програмних системах. Особливу увагу я приділив патерну Bridge, який використав у власній реалізації для відокремлення абстракції терміналу від механізмів його відображення. Використання Bridge дозволило зробити систему більш гнучкою, спростило додавання нових способів рендерингу та забезпечило незалежний розвиток обох ієрархій – логіки терміналу та засобів його виведення. Робота з цими патернами допомогла краще зрозуміти принципи побудови масштабованих і розширюваних архітектур.

Контрольні запитання

1. Яке призначення шаблону «Посередник»?

Шаблон «Посередник» зменшує зв'язність між об'єктами, виступаючи центральним вузлом комунікації. Замість того щоб об'єкти напряму викликали методи один одного, вони взаємодіють через посередника, що спрощує підтримку та розширення системи.

2. Нарисуйте структуру шаблону «Посередник».



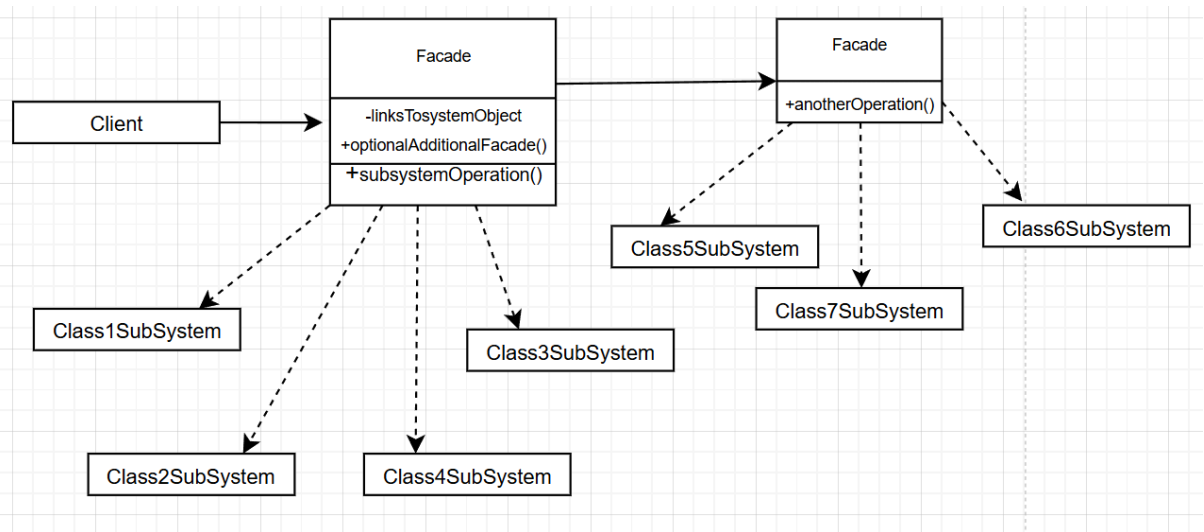
3. Які класи входять в шаблон «Посередник», та яка між ними взаємодія?

У шаблон «Посередник» входять наступні класи: Mediator – інтерфейс для комунікації між об'єктами; ProductA – конкретна реалізація посередника; Colleague – абстрактний клас або інтерфейс учасників; ConcreteColleague – конкретні об'єкти, що взаємодіють через медіатор. Колеги надсилають повідомлення медіатору, який визначає, кому і як їх передати.

4. Яке призначення шаблону «Фасад»?

Шаблон «Фасад» спрощує доступ до складної системи, надаючи єдиний інтерфейс. Він приховує деталі реалізації підсистеми та зменшує залежність клієнтського коду від внутрішньої структури системи.

5. Нарисуйте структуру шаблону «Фасад».



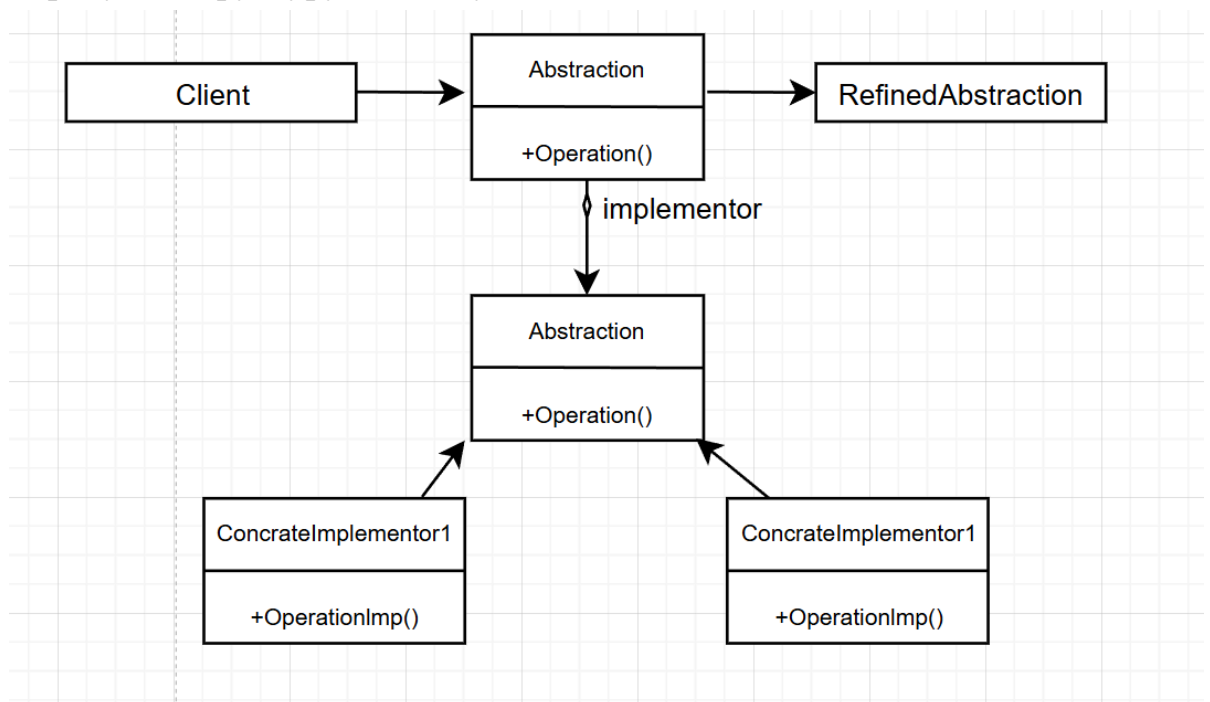
6. Які класи входять в шаблон «Фасад», та яка між ними взаємодія?

Класи шаблону «Фасад» включають **Facade** – клас, що надає єдиний інтерфейс, та **Subsystem classes** – конкретні підсистеми з власними методами. Клієнт звертається лише до фасаду, а фасад делегує виклики підсистемам, забезпечуючи спрощений доступ і приховання внутрішніх деталей.

7. Яке призначення шаблону «Міст»?

Шаблон «Міст» розділяє абстракцію і реалізацію, що дозволяє змінювати їх незалежно одна від одної. Це допомагає уникнути множинного наслідування та спрощує розширення системи.

8. Нарисуйте структуру шаблону «Міст».



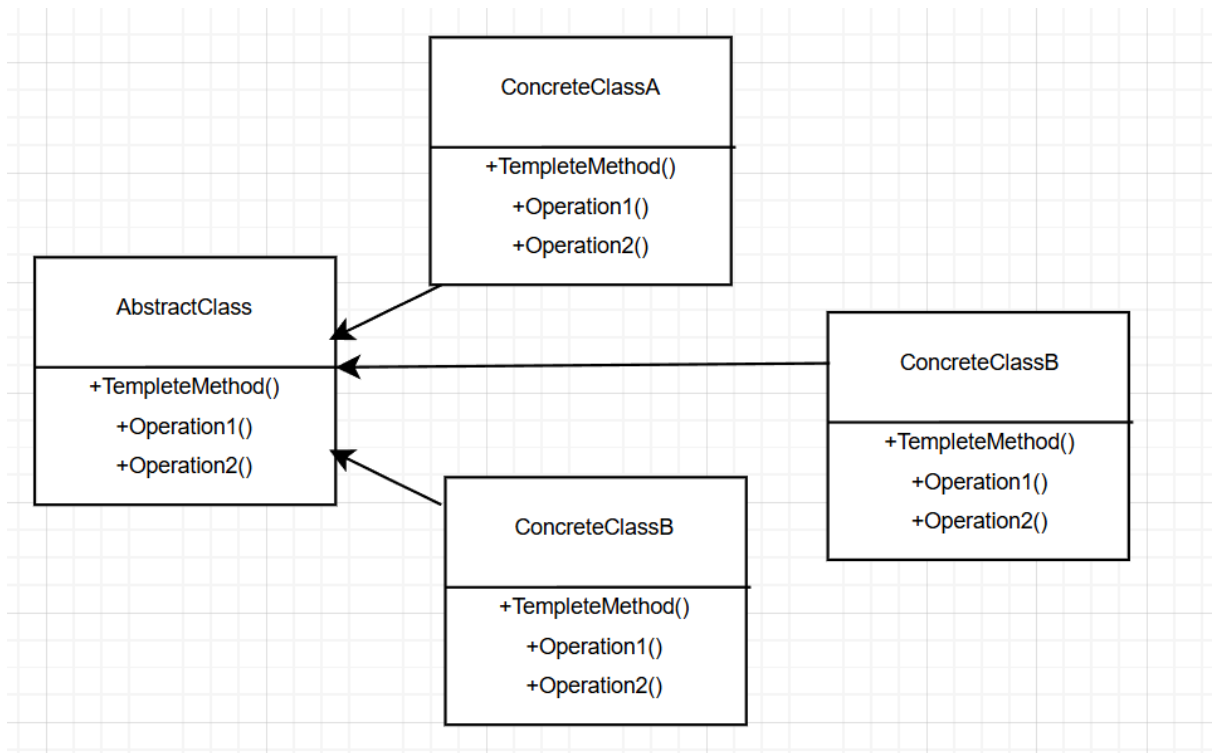
9. Які класи входять в шаблон «Міст», та яка між ними взаємодія?

Класи шаблону «Міст» включають **Abstraction** – базовий інтерфейс абстракції; **RefinedAbstraction** – розширення абстракції; **Implementor** – інтерфейс реалізації; **ConcreteImplementor** – конкретна реалізація. Абстракція делегує операції реалізатору через посилання на інтерфейс реалізації, що дозволяє змінювати обидві частини незалежно.

10. Яке призначення шаблону «Шаблонний метод»?

Шаблон «Шаблонний метод» задає каркас алгоритму в базовому класі та дозволяє підкласам перевизначати окремі кроки, не змінюючи структуру всього алгоритму.

11. Нарисуйте структуру шаблону «Шаблонний метод».



12. Які класи входять в шаблон «Шаблонний метод», та яка між ними взаємодія?

Класи шаблону «Шаблонний метод» включають AbstractClass – клас, який визначає послідовність кроків алгоритму, та ConcreteClass – клас, який реалізує конкретні кроки. Клієнт викликає шаблонний метод, що забезпечує виконання фіксованої послідовності кроків, частково визначених у підкласах.

13. Чим відрізняється шаблон «Шаблонний метод» від «Фабричного методу»?

Шаблонний метод визначає алгоритм і дозволяє змінювати його кроки в підкласах, тоді як фабричний метод визначає, який саме об'єкт створювати, змінюючи типи об'єктів, а не послідовність дій.

14. Яку функціональність додає шаблон «Міст»?

Шаблон «Міст» дозволяє розділяти абстракцію і реалізацію, незалежно розширювати обидві частини, зменшувати кількість класів, уникаючи комбінацій «абстракція × реалізація», а також полегшує підтримку та тестування коду.

